

title: ADR — KZTEK Labeling Studio Cải Tiến (Kiến trúc tổng thể)

plan: docs/plans/PLAN-labeling-studio-improve-2026-08-04/PLAN-MASTER.md

step: 0.2

author: tech-lead

approver: cto

created: 2026-08-04

updated: 2026-08-04

status: APPROVED

ADR — KZTEK Labeling Studio Cải Tiến Toàn Diện

ADR này chốt các quyết định kiến trúc mẫu chốt cho toàn bộ plan cải tiến (22 bước code, Phase 1–6). Chi tiết kỹ thuật (SQL đầy đủ, API shape, sequence) nằm ở `docs/tech-design/TDD-labeling-studio-improve.md`.

Ngữ cảnh

- KZTEK Labeling Studio hiện là bản viết lại server thay tool Tkinter cũ. Stack: Node/Express + SQLite + React 18/TS + Python (spawn `infer.py` mỗi lần).
 - Plan cải tiến gồm 6 phase: (0) Design, (1) Hạ tầng nền tảng, (2) Auth, (3) Schema mở rộng + Lịch sử, (4) Cache detect, (5) UX, (6) Nâng cao.
 - Ba nhóm mục tiêu: (a) tối ưu thời gian xử lý label, (b) lịch sử/audit rõ ràng với actor_id từ đầu, (c) tính năng nâng cao.
-

Quyết định

AD-1. Inference Python: FastAPI thường trực (localhost:8001), Node quản lý lifecycle

- Framework: **FastAPI + uvicorn**, chạy tại `127.0.0.1:8001` (env `INFERENCE_PORT`).
- Lifecycle: Node `spawn` uvicorn khi server start, `kill` khi shutdown (SIGINT/SIGTERM/exit).
- Model cache: LRU 3 model trong RAM.
- Không dùng systemd, không dùng Unix socket, không dùng gRPC. Giữ `infer.py` cũ (`USE_LEGACY_INFER=1` để rollback).
- Fallback khi service down: trả `503 INFERENCE_UNAVAILABLE`, không tự động rơi về spawn cũ (tránh 2 code path song song).

Alternatives xem xét:

- Flask (đơn giản hơn) — bỏ vì async/type safety kém hơn FastAPI.
- systemd service riêng — bỏ vì phức tạp ops, không cross-platform dev.
- Unix socket — bỏ vì Windows không hỗ trợ trực tiếp.

AD-2. Auth: JWT (HS256), bcrypt cost=12, dual-mode cookie + Bearer

- Token: JWT trong httpOnly cookie `kztek_token` (SameSite=Lax) + `Authorization: Bearer` fallback.
- TTL 24h, không refresh token trong Phase 2 (thêm khi có yêu cầu thực).
- Secret bắt buộc qua env `AUTH_JWT_SECRET` (≥ 32 ký tự), server refuse start nếu thiếu.
- Roles: `annotator` / `reviewer` / `admin`. Middleware global verify + per-route `requireRole()`.
- Rate limit login: 10 lần / IP / 15 phút (in-memory).
- Seed admin đầu tiên qua env `AUTH_BOOTSTRAP_ADMIN_*`, log warning nếu dùng default.

Alternatives xem xét:

- Session-cookie + bảng `sessions` — bỏ vì thêm state phía server, không cần với single-node scale hiện tại.
- OAuth/SSO — bỏ ở phase này, có thể thêm sau qua tầng middleware.
- Argon2 thay bcrypt — bỏ vì bcrypt cost=12 đủ, cộng đồng Node rộng hơn.

AD-3. Bảng `users`: INTEGER PK AUTOINCREMENT (khác các bảng cũ dùng TEXT nanoid)

- `users.id` `INTEGER AUTOINCREMENT` — dùng INTEGER cho user domain mới.
- FK từ `annotation_history.actor_id`, `activity_log.actor_id`, `images.completed_by`, `jobs.actor_id` = INTEGER (đồng nhất).
- Các bảng cũ (`projects`, `classes`, `images`, `annotations`, `models`) GIỮ nguyên TEXT id (nanoid).
- Cột `color` mặc định `#4A3F8C` (KZTEK sub-heading) để hiển thị actor chip trên UI history/activity.

Lý do: Không quá tải khi mix hai kiểu id; user domain hoàn toàn độc lập với entity domain nên không có nguy cơ nhầm lẫn ở business logic.

AD-4. Migration strategy: Giữ pattern inline idempotent, tách ra `server/src/migrations.js`

- KHÔNG dùng ORM (Knex/Prisma/Sequelize) hay tool migration mới (node-pg-migrate, umzug).
- Giữ pattern hiện có (`PRAGMA table_info` + `ALTER TABLE IF NOT EXISTS` thủ công), nhưng chuyển từ inline trong `db.js` sang file riêng `migrations.js` — dễ đọc, dễ audit, không đổi runtime.
- `runMigrations(db)` gọi trong `index.js` sau khi mở DB, trước khi mount route.
- Trước migration mới: tự động backup `app.db` → `app.db.bak-{YYYYMMDD-HH:mm:ss}`, giữ 5 backup gần nhất.
- Mỗi migration wrap trong `db.transaction(...)`.

Alternatives xem xét:

- ORM (Prisma/Drizzle) — bỏ vì stack hiện dùng raw SQL, thêm ORM là refactor lớn ngoài scope.
- Kysely/knex migration file numbering — bỏ vì thêm dependency + học phí, không giá trị so với hàm idempotent hiện có.

AD-5. Annotation history: SNAPSHOT toàn bộ annotations của ảnh tại mỗi save (không diff)

- Bảng `annotation_history` lưu `snapshot` TEXT (JSON array đầy đủ box của ảnh tại version).
- Ưu điểm: revert 1 lệnh (REPLACE annotations), không cần apply logic.
- Ước lượng: $\sim 1-5 \text{ KB/save} \times 100\text{K save/năm} \approx 200-500 \text{ MB}$ → chấp nhận được.
- Retention: giữ 200 version gần nhất/ảnh, xoá cũ hơn qua timer 1h.

Alternatives xem xét:

- Diff-based (patch từng box) — bỏ vì phức tạp, dễ bug apply diff, revert tốn nhiều query.

AD-6. Cột `version` trên `annotations` cho optimistic locking

- `annotations.version INTEGER NOT NULL DEFAULT 1` — tăng mỗi lần save.
- Client gửi PUT kèm `expected_version`; server so sánh:
 - Match → increment, save, insert history.
 - Mismatch → 409 Conflict + trả version + snapshot hiện tại để client resolve.
- Thêm `updated_at TEXT, updated_by INTEGER FK users(id)` cho audit inline.

AD-7. Image done status: 2 field độc lập, KHÔNG đổi enum `status`

- `images.completed_at TEXT NULL, images.completed_by INTEGER NULL FK users(id)`.
- `images.status (unlabeled/labeled)` GIỮ NGUYÊN — backend tự set theo có annotation hay không.
- Cho phép trạng thái mixed: `labeled` + chưa `completed` (chờ review), `unlabeled` + `completed` (negative sample xác nhận).
- Endpoint: reuse `PATCH /api/projects/:pid/images/:iid` với body `{completed: bool}`.

Alternatives xem xét:

- Mở rộng enum status thành `unlabeled|labeled|completed|reviewed` — bỏ vì mixed state không biểu diễn được, migration data cũ phức tạp.

AD-8. Quick class switcher: MRU 1-9 (localStorage), Ctrl+K fuzzy, hotkey 2 ký tự

- MRU: localStorage per-user per-project (`kztek.mru.{userId}.{projectId}`), tối đa 9 entries, không dùng DB.
- Fuzzy search: subsequence match tự viết, không thêm lib.
- Hotkey mở rộng: `classes.hotkey` chấp nhận 1-2 ký tự [A-Za-z0-9]. Sửa `normalizeHotkey` trong `classes.js`.

AD-9. Jobs persist: bảng `jobs` thay in-memory Map, giữ nguyên API shape client

- Bảng `jobs (id TEXT PK, project_id, kind, status, total, done, created, failed, error, payload, actor_id, timestamps)`.
- Autolabel refactor: đọc/ghi DB thay Map. Client `startAutoLabel/getAutoLabelJob` KHÔNG đổi contract.
- Bonus: `GET /api/projects/:pid/jobs` để hiển thị lịch sử job.

AD-10. Fan-out inference batch: 32 ảnh/request Node → FastAPI

- Node chia job lớn thành nhiều request `POST /predict` (32 ảnh mỗi request), tuần tự.
- Cập nhật `jobs.done` sau mỗi batch → UI polling thấy progress mượt.
- Không dùng streaming NDJSON (như `infer.py` cũ) — client polling qua `jobs` table đủ.

Hệ quả

Tích cực:

- Tránh spawn Python mỗi lần → giảm 3–8 s/lần start-up model cho auto-label.
- Actor_id trong mọi audit trail ngay từ Phase 3 → không phải backfill sau.
- Rollback đơn giản (`USE_LEGACY_INFER=1, AUTH_DISABLED=1, .db.bak-*`).
- Client contract cho auto-label / images / annotations về cơ bản không đổi → giảm rủi ro breakage UI.

Tiêu cực:

- Thêm 1 process Python luôn chạy → tốn ~300–500 MB RAM idle. Chấp nhận được.
- Mix INTEGER (users) và TEXT (entities) trong FK → dev cần chú ý khi viết query JOIN.
- `annotation_history` snapshot phình DB theo thời gian → cần retention timer.

Rủi ro trung hạn:

- Nếu scale lên nhiều node → cần thay JWT secret shared, thay in-memory rate-limit bằng Redis. Phase hiện tại chưa cần.

Security posture (nhắc trước Phase 2/3 merge)

- Toàn bộ Phase 2 (2.1, 2.2, 2.3) và Phase 3.1 BẮT BUỘC chạy `security-audit-stride` trước khi merge — đã ghi trong PLAN-MASTER.
- Điểm nóng cần review kỹ:
 - JWT secret handling, cookie flags (Secure/HttpOnly/SameSite).
 - Bcrypt cost, timing-safe compare khi login fail (không leak username tồn tại/không).
 - Rate limit login đủ chống brute force.
 - Migration Phase 3.1 KHÔNG mất dữ liệu cũ, backup được restore.
 - Batch delete cần confirm + role check.

CTO Review

Reviewer: CTO (self-review trong cùng conversation, thay mặt vai trò CTO theo yêu cầu STEP-0.2).

Ngày: 2026-08-04.

Checklist review kiến trúc chiến lược

Điểm	Đánh giá
---	---
Trade-off có hợp lý cho scope & scale hiện tại (single-node, đội nhỏ)?	✓ Có — không over-engineer (không ORM, không systemd, không refresh token)
Rollback có khả thi cho mọi thay đổi rủi ro cao?	✓ Có — <code>USE_LEGACY_INFER, AUTH_DISABLED, .db.bak-{ts}</code>
Two-Eyes principle được giữ (security-audit-stride cho Phase 2 + 3.1)?	✓ Có — nhắc rõ trong ADR + PLAN-MASTER
API contract mới không phá client hiện có?	✓ Có — auto-label / images / annotations giữ shape; auth thêm mới
Rủi ro bảo mật auth (JWT/bcrypt/rate-limit) được nhận diện?	✓ Có — AD-2 + section Security posture
Rủi ro data loss migration Phase 3.1?	✓ Có mitigation — backup auto + transaction wrap; nhưng CTO YÊU CẦU: bước 3.1 thực thi PHẢI có bước "verify count row bảng annotations trước/sau migration" trong quy trình, không chỉ dựa vào

transaction.

Quyết định

APPROVE với **1 điều kiện bổ sung** (đã ghi vào tài liệu bởi CTO):

CTO condition #1: Bước 3.1 thực thi **PHẢI**: (a) đếm số dòng **annotations, images, classes** TRƯỚC migration, (b) chạy migration trong transaction, (c) đếm lại SAU migration, (d) nếu count không khớp → rollback ngay từ backup. Bước 3.1 **KHÔNG** được merge nếu bước verify này không có trong code migration hoặc trong step file.

Ký:

- Tech Lead: đã viết TDD + ADR, đề xuất APPROVE 2026-08-04.
- CTO: **APPROVED** kèm điều kiện #1, 2026-08-04.